Vibe code app 7

Generated by scripts/build_docs_pdf.py

April 13, 2026

16.11 What the UI should stay focused on

Generated April 13, 2026

---

1 Alarm Model

2 Alarm model draft

This file defines the first-pass alarm model for app 7.

2.1 Goals

- understandable to an operator
- simple enough for an MVP
- extensible later for richer logic
- clearly separate alarm definition from alarm event

2.2 Core concepts

2.3 Alarm definition

An alarm definition describes what should be watched and when an event should fire.

Suggested fields:

- `id`
- `name`
- `enabled`
- `deviceId`
- `pointId`
- `severity` (`info`, `warning`, `critical`)
- `conditionType`
- `condition`

- `messageTemplate`
- `deadband`
- `persistenceSeconds`
- `autoClear`
- `notifyChannels`
- `createdAt`
- `updatedAt`

2.3.1 Candidate condition types

- `greaterThan`
- `greaterThanOrEqual`
- `lessThan`
- `lessThanOrEqual`
- `equal`
- `notEqual`
- `stale`
- `outOfRange`
- `boolTrue`
- `boolFalse`

2.4 Alarm event

An alarm event is a runtime occurrence created when a definition transitions into alarm.

Suggested fields:

- `id`
- `alarmDefinitionId`
- `deviceId`
- `pointId`
- `state` (`active`, `acknowledged`, `cleared`)
- `severity`
- `message`
- `triggeredAt`
- `acknowledgedAt`
- `acknowledgedBy`
- `clearedAt`
- `lastObservedValue`

- `lastObservedQuality`
- `notificationStatus`

2.5 State transitions

Basic lifecycle:

1. definition condition becomes true
2. backend waits optional persistence period
3. event becomes `active`
4. operator may acknowledge -> `acknowledged`
5. condition clears -> event becomes `cleared`

Acknowledged alarms may still remain visually active until the condition clears.

2.6 Suggested MVP rule types

Start with a small set:

- high limit
- low limit
- stale/no-update
- binary fault state
- out-of-range

This covers a lot of BAS-lite value without overbuilding.

2.7 Noise control

To avoid alarm spam:

- use persistence timers
- use deadbands for analog thresholds
- avoid repeated notifications on every scrape
- support notification-on-enter and optional reminder-on-still-active later

2.8 Notification linkage

Alarm definitions should include desired notification routing metadata, but notification delivery results belong on alarm events / notification logs.

2.9 Example definition

```
{
  "id": "alarm-zone1-temp-high",
  "name": "Zone 1 temperature high",
  "enabled": true,
  "deviceId": "zone1-vav",
  "pointId": "zone1-space-temp",
  "severity": "warning",
  "conditionType": "greaterThan",
  "condition": { "threshold": 78.0 },
  "deadband": 1.0,
  "persistenceSeconds": 300,
  "autoClear": true,
  "notifyChannels": ["smtp"]
}
```

2.10 Example event

```
{
  "id": "evt-0001",
  "alarmDefinitionId": "alarm-zone1-temp-high",
  "deviceId": "zone1-vav",
  "pointId": "zone1-space-temp",
  "state": "active",
  "severity": "warning",
  "message": "Zone 1 temperature high",
  "triggeredAt": "2026-04-11T06:15:00Z",
  "lastObservedValue": 79.3,
  "notificationStatus": "sent"
}
```

2.11 Future expansions

Later, this model can grow into:

- schedules / occupied vs unoccupied logic
- multi-point compound conditions
- delayed clear behavior
- escalation chains
- shelving/suppression
- maintenance overrides
- fault-rule templates by equipment type

3 Architecture

4 App 7 architecture - BMS/ BAS Lite

4.1 Intent

Build a slim operator-facing BAS/BMS Lite app for Raspberry Pi-class deployments, using VOLTTRON as middleware instead of using VOLTTRON Central as the product UI.

4.2 Design principles

- BACnet-first
- small-system friendly
- SD-card-aware logging and retention
- explicit API boundary between UI and runtime
- easy to teach, recreate, and evolve
- Open-FDD-inspired frontend feel without dragging in unnecessary complexity

4.3 Layered architecture

4.4 1) Frontend

A React-based UI focused on operations visibility and control.

Primary responsibilities:

- show a very simple device tree for the current bench equipment
- repopulate the main dashboard when a device is selected
- show live-ish point values / freshness / status
- show trends over short windows, ideally with Plotly-style zoom/pan/export behavior
- show alarms, alarm severity, and acknowledgement state
- keep config surfaces minimal during the bench phase
- surface basic health state for the app/runtime

For the current 2-device bench, prefer a simpler operator dashboard over a many-pane control center. Alarm/trend setup can be driven by OpenClaw chat and reflected in notes/APIs before a full UI editor exists.

Future commissioning posture should explicitly allow another OpenClaw instance to: - deploy the app on a fresh VOLTTRON Pi base - configure alarm/trend posture - guide SMTP dial-out testing with the technician present - verify writable BACnet setpoints on approved points

Frontend should talk only to the app backend, not directly to VOLTTRON.

4.5 2) App backend / API

A lightweight application service between frontend and runtime.

Primary responsibilities:

- expose stable UI-facing endpoints
- normalize device/point metadata

- manage polling state and write intent where allowed
- own alarm definitions and alarm-event lifecycle
- expose trend query endpoints
- store notification config and send history
- enforce retention/cleanup policy
- provide health/status summary for UI

This backend is the product boundary.

4.6 3) VOLTTRON layer

Primary responsibilities:

- BACnet Proxy
- Platform Driver
- scrape/publish pipeline
- supervisory agents
- reusable building/process logic

The VOLTTRON layer should be treated as runtime integration infrastructure, not the final operator-facing application.

4.7 Data flow

1. BACnet devices are discovered/configured through the VOLTTRON integration path.
2. Driver/scrape paths publish current values into the runtime.
3. The app backend ingests, caches, or queries current values as needed.
4. Alarm evaluation uses backend-owned definitions with VOLTTRON-backed point data.
5. UI reads the backend API for inventory, current state, alarms, trends, and notification history.

4.8 Storage posture

Separate storage classes on purpose:

4.8.1 Current-state cache

- latest point values
- device freshness / heartbeat
- runtime health summary
- keep lightweight and replaceable
- current bench preference: in-memory

4.8.2 Short-term trends

- recent time series for operator graphs
- bounded retention
- likely rollups/downsampling later
- current bench preference: in-memory bounded cache sized around a month-scale target, not unbounded disk writes

4.8.3 Alarm/event history

- active alarms
- transitions
- acknowledgements
- notification attempts/results
- current bench preference: lightweight/in-memory until the write policy is worth hardening

4.8.4 Config/state

- device metadata overrides
- polling enabled/disabled state
- alarm definitions
- retention config
- SMTP / notification config
- current bench preference: simple/static or lightweight state before introducing a heavier DB layer

4.9 Raspberry Pi / SD-card posture

To reduce wear:

- avoid unbounded local logging

- keep verbose/debug logs off by default
- prefer bounded retention and rotation
- document journald limits explicitly
- distinguish operator history from developer/debug history
- keep trend retention configurable by deployment size

4.10 MVP operator workflows

4.10.1 Inventory view

- see devices
- expand devices into points
- see comm status/freshness
- enable/disable polling where appropriate

4.10.2 Alarm workflow

- define alarm rules
- view active alarms
- acknowledge alarms
- inspect event history
- review notification attempts

4.10.3 Trend workflow

- select point
- select recent time window
- view short-term trend
- later compare multiple points if needed

4.11 Suggested implementation order

1. define backend contract
2. build frontend shell with mock data
3. define alarm schema/state model
4. implement backend inventory + alarm endpoints
5. implement short-term trend path
6. wire real VOLTTRON-backed data in carefully

4.12 Open questions to settle later

- exact backend framework (FastAPI is a natural fit, but not mandatory)
- exact frontend component stack
- whether trend storage should start as SQLite-only or use a separate TS store
- whether alarm evaluation lives entirely in backend or partly in VOLTTRON agents
- whether writes/commands are in scope for MVP or read-mostly first

5 Backend Contract

6 Backend contract draft

This is the UI-facing contract for app 7.

The purpose is to keep the frontend stable even if the VOLTTRON-side integration evolves.

6.1 Principles

- frontend talks only to backend
- backend returns operator-friendly shapes
- backend owns alarm/event semantics
- backend normalizes data freshness/status
- API should be small before it becomes clever

6.2 Core resources

- `/api/health`
- `/api/devices`
- `/api/devices/:deviceId`
- `/api/points`

- `/api/points/:pointId`
- `/api/polling`
- `/api/alarms/definitions`
- `/api/alarms/events`
- `/api/trends`
- `/api/notifications/config`
- `/api/notifications/logs`
- `/api/setpoints`
- `/api/setpoints/write`

6.3 Health

6.3.1 `GET /api/health`

Returns:

- app status
- backend uptime
- last successful VOLTTRON sync / scrape visibility
- counts for devices, points, active alarms
- storage/retention summary

Example:

```
{
  "status": "ok",
  "backendTime": "2026-04-11T06:00:00Z",
  "volttron": {
    "status": "connected",
    "lastSync": "2026-04-11T05:59:52Z"
  },
  "counts": {
    "devices": 2,
    "points": 24,
    "activeAlarms": 1
  }
}
```

6.4 Devices

6.4.1 `GET /api/devices`

List devices for the device tree.

Example fields:

- `id`
- `name`
- `network`
- `address`
- `status`
- `lastSeen`
- `pollingEnabled`
- `pointCount`

6.4.2 `GET /api/devices/:deviceId`

Returns one device plus point summaries.

6.5 Points

6.5.1 `GET /api/points`

Filterable point list for point table.

Suggested filters:

- `deviceId`
- `kind`
- `status`
- `alarmState`
- `search`

Point fields:

- `id`
- `deviceId`
- `name`

- `objectType`
- `objectInstance`
- `units`
- `value`
- `quality`
- `lastUpdated`
- `alarmState`
- `trendEnabled`

6.5.2 `GET /api/points/:pointId`

Returns full point detail.

6.6 Polling state

6.6.1 `GET /api/polling`

Returns polling status/summary for devices and points.

6.6.2 `PATCH /api/polling/devices/:deviceId`

Example body:

```
{ "pollingEnabled": true }
```

6.6.3 `PATCH /api/polling/points/:pointId`

Example body:

```
{ "pollingEnabled": false }
```

6.7 Alarm definitions

6.7.1 `GET /api/alarms/definitions`

Lists alarm rules.

6.7.2 `POST /api/alarms/definitions`

Creates a new rule.

6.7.3 `PATCH /api/alarms/definitions/:alarmDefinitionId`

Updates a rule.

6.7.4 `DELETE /api/alarms/definitions/:alarmDefinitionId`

Disables/removes a rule depending on product choice.

6.8 Alarm events

6.8.1 `GET /api/alarms/events`

Lists active/recent alarm events.

Suggested filters:

- `state=active|cleared|acked`
- `severity`
- `deviceId`
- `pointId`
- `from`
- `to`

6.8.2 `POST /api/alarms/events/:eventId/ack`

Acknowledges an alarm event.

6.9 Trends

6.9.1 `GET /api/trends`

Query parameters:

- `pointId`
- `from`

- `to`
- `bucket`

Response should return a small, chart-friendly time-series payload.

6.10 Notifications

6.10.1 `GET /api/notifications/config`

6.10.2 `PUT /api/notifications/config`

Store SMTP/notification config.

6.10.3 `GET /api/notifications/logs`

List notification attempts/results.

Fields might include:

- `timestamp`
- `channel`
- `recipient`
- `eventId`
- `status`
- `error`

6.11 Initial implementation guidance

For the first pass:

- build these routes as live in-memory-backed endpoints over the 2-device bench data
- keep response shapes stable
- avoid overcommitting to internal implementation too early
- allow OpenClaw chat to act as the practical operator/config layer for alarm/trend setup during early bench work
- keep the browser UI lighter and simpler than the API surface

7 Bosspi Volttron Dataflow And Storage Tutorial

8 bosspi VOLTTRON + BACnet + App 7 tutorial

Practical handoff for humans and AI assistants: how the Raspberry Pi bench is wired, how BACnet data reaches the App 7 web UI, what lives on disk versus memory, how to run VOLTTRON under **systemd** safely, and how to avoid **SD card wear** on a Pi.

Scope: standalone **edge** VOLTTRON on `bosspi` with App 7. This document intentionally does **not** cover VOLTTRON Central, ForwardHistorian, or Open-FDD—those are separate integration tracks.

8.1 Quick context (for AI assistants)

Use this block as grounding when editing agents or ops runbooks.

Item	Value
Edge host	`bosspi` (`192.168.204.12`), user `ben`
VOLTTRON source / venv	`/home/ben/volttron`, activate `source env/bin/activate`
`VOLTTRON_HOME`	`/home/ben/.volttron`
systemd unit	`volttron.service`
App 7 URL	`http://192.168.204.12:8080/app7/index.html`
BACnet bench devices	`BensFakeAHU` @ `192.168.204.13`, `Zone1VAV` @ `192.168.204.14`
Data path (mental model)	

Item	Value
	BACnet → BACnet proxy → Platform Driver → topics → App 7 web agent → browser
Pi constraints	Prefer **memory-first** telemetry and **bounded** logs; avoid unbounded disk logging

Canonical upstream docs: VOLTTRON index, web framework.

8.2 1. Big picture

One primary system for this tutorial:

1. **bosspi (`192.168.204.12`)** — Raspberry Pi edge runtime: native VOLTTRON in `~/volttron`, BACnet to bench devices, App 7 web agent.

Bench BACnet devices

- `BensFakeAHU` → `192.168.204.13`
- `Zone1VAV` → `192.168.204.14`

End-to-end flow

BACnet devices → BACnet proxy → Platform Driver on bosspi → topic bus → App 7 web agent → browser

---

8.3 2. Core online docs

- Main index: https://volttron.readthedocs.io/en/main/index.html
- Web framework: https://volttron.readthedocs.io/en/main/agent-framework/web-framework.html

Typical hard parts on this bench:

1. Reliable BACnet/IP on the Pi
2. Healthy VOLTTRON under **systemd** (no duplicate manual + service starts)

3. App 7 served correctly through the platform web service

8.4 3. Python version vs VOLTTRON

Rule: match Python to the **exact** VOLTTRON stack you installed on the Pi.

- **Modular VOLTTRON** (current PyPI `volttron` / Eclipse direction): **Python ≥ 3.10** (PyPI metadata is `>=3.10,<4`; upstream docs target **3.10** as the tested baseline). **3.11** is often fine—confirm with your installed package and tests on the Pi.
- **Older monolithic VOLTTRON** (e.g. 8.x clones): may require an older Python; do not assume 3.10+ without reading that release's install guide.

On the Pi, always verify:

```
cd /home/ben/volttron
source env/bin/activate
python3 --version
pip show volttron | sed -n '1,12p'
```

If you upgrade Python, recreate the venv and reinstall VOLTTRON and agents.

8.5 4. bosspi setup at a glance

8.5.1 Host and paths

- Hostname: `bosspi`
- SSH: `ben@192.168.204.12`
- VOLTTRON repo: `/home/ben/volttron`
- `VOLTTRON_HOME`: `/home/ben/.volttron`
- systemd: `volttron.service`

8.5.2 Shell prelude (every VOLTTRON CLI session)

```
cd /home/ben/volttron
export VOLTTRON_HOME=/home/ben/.volttron
source env/bin/activate
```

8.5.3 Agents commonly involved on the bench

- `platform.bacnet_proxy`
- `platform.driver`
- `listener.bacnet` (if used)
- Custom bench agents (e.g. GL36 family) as configured
- `ben.app7.web` — App 7 web agent

8.6 5. How BACnet data reaches the frontend

8.6.1 Step A — BACnet proxy

Speaks BACnet/IP to devices: discovery, reads, writes; backs Platform Driver.

8.6.2 Step B — Platform Driver

Maps devices/points to publishable topics, for example:

- `devices/BensFakeAHU/all`
- `devices/Zone1VAV/all`

8.6.3 Step C — Consumers

Listener agents, logic agents, **App 7 web agent**, etc., subscribe or query through the platform.

8.6.4 Step D — App 7 web agent

Serves UI and app routes via VOLTTRON's web framework; the browser does **not** speak BACnet.

Browser path:
browser → VOLTTRON web service → App 7 web agent → runtime/topics → Platform Driver → BACnet proxy → devices

8.6.5 Step E — Why URLs look like VOLTTRON

Agents register routes and static files; the platform web service hosts them—no separate nginx/node stack required for App 7 on this bench.

8.7 6. Local storage posture on the Pi

8.7.1 Live telemetry

Source of truth for **live** values is runtime: BACnet reads, driver publications, agent state—not a large App-specific SQL store.

8.7.2 App 7

Posture: **memory-first / bounded** for high-churn dashboard and trend buffers; avoid always-growing local databases for UI telemetry.

8.7.3 `VOLTTRON_HOME` on disk

Even when apps are memory-friendly, the platform keeps control data under `/home/ben/.volttron` (configs, keystores, agent installs, **some logs** depending on setup). So "everything in RAM" is not literal—**high-churn telemetry** should be RAM-first; **platform state** stays where VOLTTRON expects it.

8.8 7. Logging and SD card wear

8.8.1 Reality check

Logs may appear in:

1. Files under `VOLTTRON_HOME` or the repo (e.g. `volttron.log`)
2. **journald** via `volttron.service`
3. Custom agent or script file logging

8.8.2 Design goals on SD card storage

- **No high-rate, unbounded append logging** to the card for telemetry or per-sample traces.
- **INFO/WARN** in normal operation; **DEBUG** only while troubleshooting.
- **Bounded** retention everywhere something is allowed to grow.

8.8.3 Prefer logs in memory (journald), not endless files on the card

systemd journal to RAM (strong Pi pattern): persist logs only when you need post-reboot forensics. Example drop-in:

```
sudo mkdir -p /etc/systemd/journald.conf.d
sudo tee /etc/systemd/journald.conf.d/volatile-storage.conf >/dev/
        null <<'EOF'
[Journal]
Storage=volatile
RuntimeMaxUse=64M
EOF
sudo systemctl restart systemd-journald
```

Tradeoff: journal contents are lost on reboot. For long investigations, temporarily switch to persistent storage or copy logs off-device.

Cap persistent journal (if you keep `Storage=persistent`):

```
[Journal]
SystemMaxUse=100M
MaxRetentionSec=1week
```

Pair with routine checks:

```
journalctl --disk-usage
```

8.8.4 VOLTTRON file logs

If `volttron.log` grows without rotation, fix it: logrotate, platform logging settings, or route platform stdout/stderr through systemd only (see service template below) and avoid duplicate huge file sinks.

8.8.5 Other Pi practices that reduce SD wear

- **tmpfs** for `/tmp` (often default on Raspberry Pi OS).
- **ZRAM** or careful **swap** policy—avoid thrashing swap on the card.
- **noatime** mounts where appropriate.
- Move heavy databases or historians to **USB SSD** or another host if you add them later.
- **Read-only root** is an advanced option for fixed appliances; plan before enabling.

8.8.6 Commands to inspect growth

```
systemctl status volttron.service --no-pager
journalctl -u volttron.service -n 100 --no-pager
ls -lah /home/ben/.volttron
find /home/ben/.volttron -type f -printf '%s %p\n' 2>/dev/null |
        sort -nr | head -30
```

8.9 8. systemd: good service posture

8.9.1 Two modes—never mix casually

1. **Service mode** — `systemctl start volttron.service`; use `journalctl` / `systemctl status`.
2. **Manual debug** — `sudo systemctl stop volttron.service` first, then activate the venv and start the platform manually.

Running both causes confusing failures (stale VIP sockets, double binds, odd `vctl status`).

8.9.2 Common commands

```
sudo systemctl status volttron.service --no-pager
sudo systemctl stop volttron.service
sudo systemctl start volttron.service
sudo systemctl restart volttron.service
sudo systemctl enable volttron.service
journalctl -u volttron.service -n 100 --no-pager
```

8.9.3 Example `volttron.service` unit

Adapt `User`, `Group`, and paths to your Pi. This pattern exports `VOLTTRON_HOME`, uses the venv's `volttron` executable, sends **stdout/stderr to the journal**, and puts the optional **`-l` file log under `/run`** (RAM, cleared on reboot) so routine platform logging does not wear the SD card.

```
[Unit]
Description=VOLTTRON platform (bosspi)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=ben
Group=ben
RuntimeDirectory=volttron
WorkingDirectory=/home/ben/volttron
```

```
Environment=VOLTTRON_HOME=/home/ben/.volttron
ExecStart=/home/ben/volttron/env/bin/volttron -vv -l /run/
        volttron/volttron.log
Restart=on-failure
RestartSec=5

LimitNOFILE=65535

StandardOutput=journal
StandardError=journal
SyslogIdentifier=volttron

[Install]
WantedBy=multi-user.target
```

Notes:

- **`RuntimeDirectory=volttron`** creates `/run/volttron` for this service; the **`-l`** path keeps a traditional log file **in memory**, not on the card. For deep postmortems across reboots, temporarily log to a **rotated** path on disk or raise journal persistence—then revert.
- Install with `sudo cp volttron.service /etc/systemd/system/`, then `sudo systemctl daemon-reload` and `sudo systemctl enable --now volttron.service`.
- Version the real unit under the repo if you want deploys reproducible (optional).

8.10 9. BACnet bring-up ladder

Order matters:

1. Pi platform healthy (systemd + `vctl status`)
2. BACnet proxy healthy
3. Platform Driver healthy and topics publishing
4. App 7 endpoints return data
5. Frontend renders

Skipping straight to the UI wastes time when the field bus is the real fault.

8.11 10. Commands worth keeping handy

8.11.1 On bosspi

```
cd /home/ben/volttron
export VOLTTRON_HOME=/home/ben/.volttron
source env/bin/activate
vctl status
```

8.11.2 Browser

- `http://192.168.204.12:8080/app7/index.html`

8.11.3 Optional: port 80 “front door” with Caddy

If you want **`http://<pi>/`** to land on the web agent (instead of VOLTTRON’s default **`/index.html`** on **8080**), use the **Caddy** pack in **`vibe_code_apps_8/caddy/`**: reverse proxy to **`127.0.0.1:8080`**, redirect **`/`** → **`/app7/index.html`** or **`/app8/index.html`**, optional **Basic Auth** and **TLS**, driven by **`/etc/default/caddy-bas-lite`**. See **`vibe_code_apps_8/docs/bas-lite-app8-tutorial.md`** §7 and **`vibe_code_apps_8/caddy/README.md`**.

8.12 11. Next-round checklist (optional)

- Record `pip show volttron` and `python3 --version` in your ops notes.
- Version the real `/etc/systemd/system/volttron.service` next to this repo if desired.
- Document BACnet driver config files for the two bench devices.
- Audit disk growth quarterly after any new agents or historians.

8.13 12. Bottom line

- **bosspi** is the edge runtime.
- **BACnet proxy + Platform Driver** ingest device data.
- **App 7** is a VOLTTRON-hosted web agent on top.
- **High-churn data** stays **memory-first and bounded**; **logs** should default to **bounded or RAM-backed journal**, not silent SD death by megabytes per hour.
- **systemd** owns production starts; stop it before manual debugging.
- **Python 3.10+** for current modular stacks—confirm against the installed `volttron` package on the Pi.

That is the architecture story for this folder.

9 Completed Codebase Map

10 Completed codebase map

This file is for humans and future OpenClaw/model sessions.

It answers one question:

Where is the completed app-7 codebase we actually care about?

10.1 Current completed codebase

10.1.1 Backend / VOLTTRON web agent

- `volttron_data/ben_bacnet/app7_web_agent/setup.py`
- `volttron_data/ben_bacnet/app7_web_agent/MANIFEST.in`
- `volttron_data/ben_bacnet/app7_web_agent/config`
- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/agent.py`

- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/__init__.py`

10.1.2 Frontend

- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/index.html`
- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/app.js`
- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/styles.css`

10.1.3 Deployment / operator docs

- `README.md`
- `RECREATE.md`
- `deploy-app7-to-bosspi.ps1`
- `docs/tech-setup-cheatsheet.md`
- `docs/model-context-notes.md`
- `docs/backend-contract.md`
- `docs/architecture.md`
- `docs/alarm-model.md`
- `docs/hosting-options.md`

10.2 What this means

If a future session wants to understand or modify app 7, it should start with the files above.

Do not hunt for old snapshots first. Do not treat deleted archive material as required context. Do not assume there is another hidden “real” codebase elsewhere in this folder.

10.3 Human summary

This folder is supposed to be:

- current code
- current docs

- lessons learned
- recreate/deploy commands

Not:

- backup museum
- stale log dump
- many conflicting copies of app 7

11 Hosting Options

12 Hosting options

12.1 Current choice

Start with **option 1**:

- serve the BAS Lite UI from a VOLTTRON web-enabled agent
- expose lightweight JSON endpoints from the same agent
- stop there and judge whether the approach feels clean enough

12.2 Why option 1 first

- lowest moving-part count
- fastest path to a Pi-hosted MVP
- honest test of whether VOLTTRON-native web serving is good enough
- avoids prematurely building a more split deployment shape

12.3 Bail criteria for later move to option 2

Move to separately hosted frontend/backend later if any of these become annoying enough:

- static asset packaging/redeploy friction
- path-prefix routing ugliness
- auth/session awkwardness
- poor frontend dev loop
- too much product logic pushed into VOLTTRON only because it can host pages

12.4 Current posture

Go hard on option 1 now, then stop and evaluate.

13 Model Context Notes

14 App 7 model-context notes

This file is the **big stuff only** context for future OpenClaw/model sessions.

14.1 Primary source-of-truth code

The codebase that matters is:

- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/agent.py`
- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/index.html`
- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/app.js`

- `volttron_data/ben_bacnet/app7_web_agent/app7_web_agent/webroot/app7/styles.css`

Ignore old prototype patterns outside that path.

14.2 Current app shape

- VOLTTRON-hosted BAS Lite app on `bosspi`
- route: `http://192.168.204.12:8080/app7/index.html`
- sidebar intentionally simple:
  - theme toggle
  - equipment tree
- main dashboard repopulates for selected device
- Plotly trend view
- simple setpoint write box
- alarms/trends/high-low limits can be configured via OpenClaw chat instead of heavy UI forms

14.3 Pi / service posture

- Pi host: `bosspi`
- main Linux service: `volttron.service`
- VOLTTRON web bind enabled at `http://192.168.204.12:8080`
- app 7 runs as agent `ben.app7.web`
- core dependencies expected healthy:
  - `platform.driver`
  - `platform.bacnet_proxy`
  - listener
  - GL36 agents

14.4 Bench BACnet devices

Current known devices:

- `BensFakeAHU` → `192.168.204.13` / device `3456789`
- `Zone1VAV` → `192.168.204.14` / device `3456790`

14.5 Data / retention posture

Current intent is Pi-friendly / SD-card-friendly:

- dashboard and trend state are bounded/in-memory first
- current target shape:
  - 5-minute trend handling
  - 31-day retention goal
- this is **not** a finished historian DB yet
- file logging should be deliberate, not noisy by accident
- advanced users may prefer journald/systemd controls for retention/rotation

14.6 Proven capabilities

- live BACnet-backed dashboard data from the 2 bench devices
- Plotly trend rendering
- real writable BACnet setpoint path via Platform Driver
- verified example: `Zone1VAV::ZoneCoolingSpt` written successfully through app 7 / VOLTTRON / Platform Driver

14.7 Alarm / trend / SMTP posture

For this bench phase, OpenClaw chat is an acceptable operator/config surface for:

- high limits
- low limits
- alarm enable/disable
- retention changes
- SMTP dial-out setup/testing

This is cleaner than forcing half-built config forms into the UI.

14.8 Performance lessons

The browser lag / `ERR_INSUFFICIENT_RESOURCES` issue was not just “slow Pi”.

It was heavily driven by frontend behavior causing overlapping request bursts.

14.8.1 What helped

- remove click-time loading splash after initial load
- cache static-ish data client-side
- keep selected device/point in JS state
- use selective refresh instead of full-page rebuild logic
- use `Plotly.react()` instead of recreating the chart every click
- dedupe trend/setpoint fetches
- allow only one background refresh at a time
- no-op clicks should not refetch
- batch renders

14.9 What not to do

14.9.1 Do not do these again

- do **not** hammer VOLTTRON web service with many overlapping endpoint calls on every click
- do **not** rebuild the whole app shell on trivial interactions unless necessary
- do **not** let background refresh stomp form inputs while the user is typing
- do **not** leave old prototype code paths in the main app-7 directory if they are no longer source-of-truth
- do **not** trust a redeploy without checking the **actual installed Pi file** when behavior seems unchanged
- do **not** assume browser cache is innocent; hard refresh (`Ctrl+F5`) matters after JS changes
- do **not** leave duplicated/junk JS tails in `app.js`; this caused multiple browser syntax failures during iteration
- do **not** overbuild operator config UI for this 2-device bench if OpenClaw chat is the simpler tool

14.10 Known rough edges / next clean step if needed

If frontend request pressure still misbehaves after caching/single-flight fixes, the next strong move is:

- create one combined dashboard endpoint for startup/device refresh

That would reduce request fan-out further and simplify the browser logic.

14.11 Caddy / reverse proxy note

Caddy was discussed as a cleaner root-path entry option so `http://192.168.204.12/` could forward to app 7, but that was **not fully implemented yet** in this workstream. Current guaranteed working path remains:

- `http://192.168.204.12:8080/app7/index.html`

15 Tech Setup Cheatsheet

16 Tech setup cheatsheet - BMS Lite on VOLTTRON + OpenClaw

This file is for future OpenClaw sessions and real technicians commissioning a Pi-hosted BAS Lite bench.

16.1 Goal

Use OpenClaw + VOLTTRON to stand up a lightweight BACnet BMS Lite instance that provides:

- device dashboard
- trends
- alarms
- SMTP dial-out test posture
- writable approved setpoints

16.2 Current bench assumptions

- Pi host: `bosspi`
- platform service: `volttron.service`
- web bind: `http://192.168.204.12:8080`
- app path: `http://192.168.204.12:8080/app7/index.html`
- BACnet devices:
  - `BensFakeAHU` @ `192.168.204.13` / device `3456789`
  - `Zone1VAV` @ `192.168.204.14` / device `3456790`

16.3 What OpenClaw should handle well

Another OpenClaw instance should be able to:

1. verify the Pi service base is healthy
2. verify BACnet Proxy + Platform Driver are healthy
3. copy/install/reinstall `app7_web_agent`
4. validate trends/alarms/endpoints
5. help configure alarm posture in notes/APIs
6. help configure SMTP dial-out and run a technician-guided test
7. verify approved writable BACnet setpoints

16.4 Core commands

```
ssh ben@192.168.204.12
cd /home/ben/volttron
export VOLTTRON_HOME=/home/ben/.volttron
```

```
source env/bin/activate
systemctl status volttron.service --no-pager
vctl status

curl http://192.168.204.12:8080/app7/api/health
curl http://192.168.204.12:8080/app7/api/devices
curl "http://192.168.204.12:8080/app7/api/points?
        deviceId=Zone1VAV"
curl "http://192.168.204.12:8080/app7/api/setpoints?
        deviceId=Zone1VAV"
```

16.5 Agent install/reinstall

```
vctl remove --tag ben-app7-web
vctl install --vip-identity ben.app7.web --tag ben-app7-web \
  /home/ben/volttron/volttron_data/ben_bacnet/app7_web_agent \
  --config /home/ben/volttron/volttron_data/ben_bacnet/
        app7_web_agent/config
vctl start --tag ben-app7-web
vctl status
```

16.6 Safe writable-point verification

Only test approved adjustable points.

Known current adjustable examples: - `BensFakeAHU::SAT_SP` - `Zone1VAV::ZoneCoolingSpt` - `Zone1VAV::VAVFlowSpt`

Example API write through app 7:

```
python3 - <<'PY'
import json, urllib.request
req = urllib.request.Request(
    'http://192.168.204.12:8080/app7/api/setpoints/write',
    data=json.dumps({'pointId': 'Zone1VAV::ZoneCoolingSpt',
        'value': 73.0}).encode(),
    headers={'Content-Type': 'application/json'},
    method='POST'
)
print(urllib.request.urlopen(req, timeout=20).read().decode())
PY
```

16.7 SMTP dial-out testing posture

Current app 7 SMTP config is placeholder/default only.

Commissioning expectation: - technician provides SMTP host/port/from/to/test target - OpenClaw updates notes/config posture - OpenClaw helps run a safe test - result is documented in notes/logs

16.8 Data retention / logging posture

Current bench posture is intentionally SD-card-friendly:

- high-churn dashboard/trend state is held in memory
- trend buffers are bounded
- current target shape is 5-minute trend handling with a 31-day retention goal
- file persistence should be added deliberately, not accidentally by noisy debug writes

16.9 Linux service posture

Preferred posture: - `volttron.service` is the main long-running service - app 7 runs as an agent inside that platform - use bounded logging - advanced users can prefer `systemd-journald` controls for retention/rotation - VOLTTRON startup helpers may support rotating-log options, but journald/systemd controls are usually the clearer ops posture on Linux

16.10 Quick log checks

```
tail -n 120 /home/ben/volttron/volttron.log

grep -n -E 'app7|Traceback|ERROR|Exception' /home/ben/volttron/
        volttron.log | tail -n 80
```

16.11 What the UI should stay focused on

- theme toggle
- equipment tree
- one repopulated dashboard per selected device
- Plotly trend with zoom/export/full-screen option
- writable setpoints for approved points only
- keep heavy config workflows out of the dashboard until truly needed